

Mutation Testing

& test effectiveness

Coverage lies. Mutation testing tells you whether your tests can actually catch bugs.

Theory · Operators · Cost · Tools + live demo · AI

What this seminar covers

From the idea, through the theory and cost, to real tools and an AI-first workflow — eShop throughout.

1 • Foundations

coverage vs sensitivity · the mutation score.

2 • Theory & history

competent programmer · coupling ·
weak/strong · HOM.

3 • Operators

the catalog of code mutations, with eShop
examples.

4 • Cost & scale

equivalent mutants · do fewer/smarter/faster ·
Google.

5 • Tools + live demo

Stryker · PIT · mutmut — a real run on eShop.

6 • AI-first

LLMs draft assertions & flag equivalents — you
verify.

Coverage lies

100% line coverage can still catch zero bugs — coverage proves execution, not sensitivity.

COVERED, BUT USELESS

```
// eShop — a test with 100% coverage...  
test("total runs", () => {  
  orderTotal([{price:100,qty:2}],{pct:10});  
  // ...but NO assertion on the result  
});
```

What coverage misses

Executed ≠ checked — a line runs with no assertion.

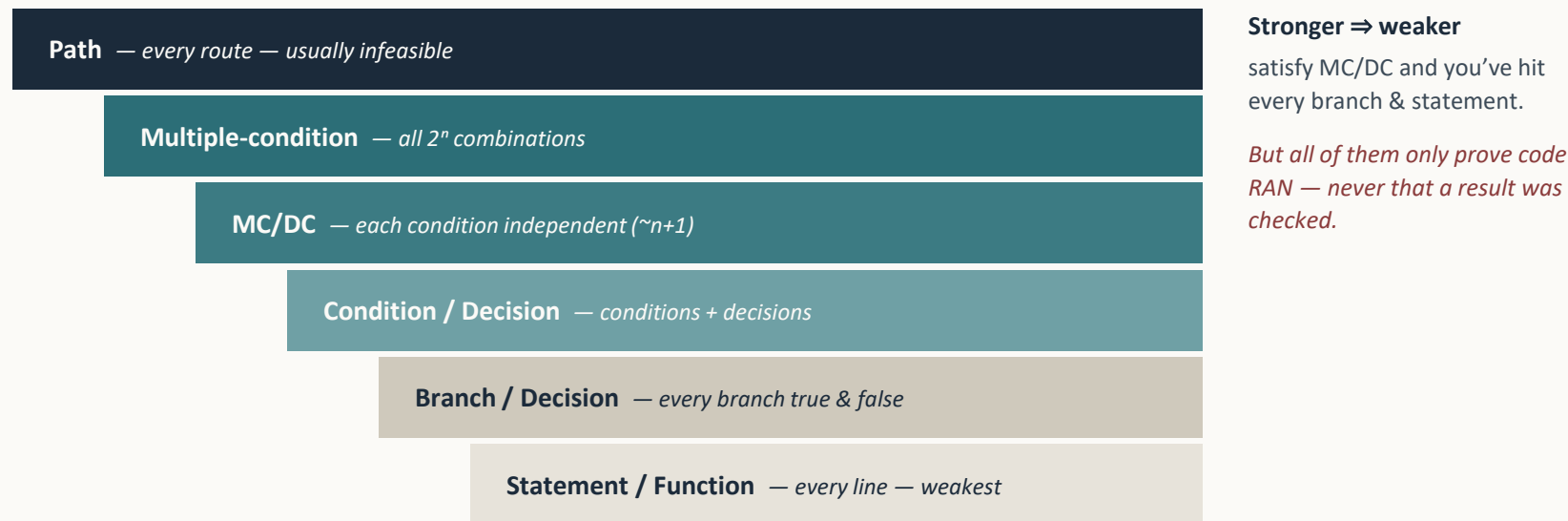
Weak asserts — `toBeDefined()` passes for wrong values.

Green ≠ safe — the suite may catch nothing at all.

THE QUESTION if I break the code on purpose, does any test notice? That is exactly what mutation testing measures.

What ‘coverage’ actually measures

Coverage has levels — each stronger criterion needs more tests. Every level proves execution, not assertions.



Coverage vs mutation testing

Two different questions — mutation measures exactly what coverage structurally cannot.

Aspect	Code coverage	Mutation testing
Question	did tests RUN the code?	would tests CATCH a bug?
Measures	execution (line/branch/MC-DC)	test sensitivity (fault detection)
Metric	% elements executed	mutation score = killed / total
Blind to	missing / weak assertions	equivalent mutants (noise)
Cost	cheap — one test run	expensive — per-mutant runs
Tools	JaCoCo · Istanbul · coverage.py	PIT · Stryker · mutmut

THE GAP IS REAL a suite at 87% coverage scored 61% mutation — the 26-point gap is executed-but-unchecked behaviour.

A test can cover every line and check nothing

A real Pitest example (RangeValidator): line AND branch coverage are 100% — yet the tests assert nothing.

SUT + A TEST THAT CHECKS NOTHING

```
boolean isValid(int input){  
    return input > 0 && input <= 100; }  
// step-0: a "test" that asserts nothing  
@Test void fifty() {  
    cut.isValid(50); // no assertion!  
JaCoCo/Sonar: line 100% · branch 100%
```

MUTATION FINDS WHAT COVERAGE CANNOT

```
// step-1: add asserts (50,200,-10)  
coverage STILL 100% — Pitest disagrees:  
Boundary (> 0) -> SURVIVED  
Boundary (<= 100) -> SURVIVED  
// step-2: add the limit cases  
isValid(100), isValid(0) -> KILLED
```

RUN IT `mvn test pitest:mutationCoverage` → `target/pit-reports`. Coverage read 100% the whole time; only mutation exposed the untested boundaries at 0 and 100.

Inject a bug, see if a test fails

A mutant is a small deliberate defect. If a test fails, the mutant is 'killed' — good.

Coverage

Measures which lines / branches ran.

Answers did a test touch this code?

Blind to whether it checked anything.

Mutation

Measures test SENSITIVITY to defects.

Answers would a bug here be caught?

Metric mutation score = killed / total.

IN ONE LINE coverage = 'did the test run the code?' · mutation = 'would the test catch a bug in that code?'

The Triangle classifier — coverage vs mutation

The classic example. Five tests reach 100% statement AND branch coverage — but do they actually check the logic?

SUT — classify a triangle

```
String triangle(int a,int b,int c){  
  if (a<=0 || b<=0 || c<=0) return "Invalid";  
  if (a+b<=c || a+c<=b || b+c<=a)  
    return "NotATriangle";  
  if (a==b&&b==c) return "Equilateral";  
  if (a==b || b==c || a==c) return "Isosceles";  
  return "Scalene"; }
```

WEAK SUITE — 100% COVERAGE

```
triangle(2,2,2) == Equilateral  
triangle(2,2,3) == Isosceles  
triangle(3,4,5) == Scalene  
triangle(1,2,9) == NotATriangle  
triangle(0,1,1) == Invalid  
coverage.py: 100% stmt · 100% branch
```

THE TRAP five tests run every line and every branch — the coverage report says ‘done’. Mutation asks a harder question: would a bug survive?

100% coverage, 67% mutation — two survivors

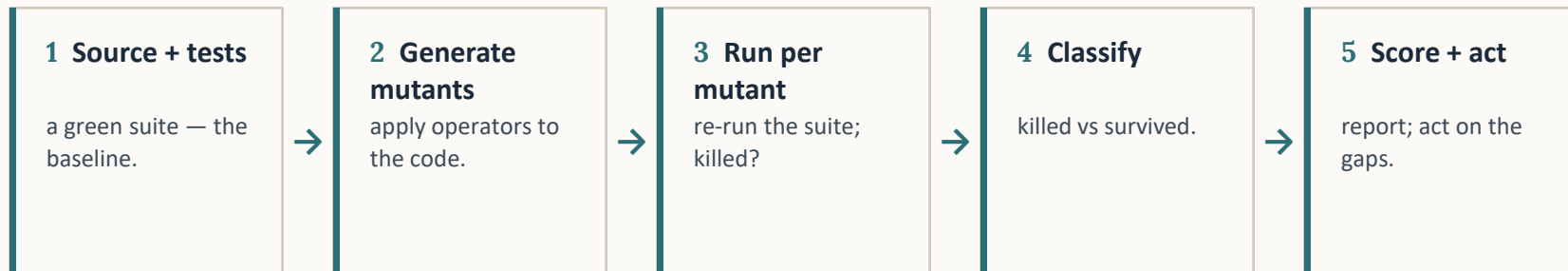
Inject 6 mutants into the same code. Four die, two survive — and each survivor is a real test gap.

Mutant (one tiny edit)	Where	Verdict
M1 $a+b \leq c \rightarrow a+b < c$	triangle-inequality boundary	SURVIVED
M2 $(a==b \ \&\& \ b==c) \rightarrow $	equilateral condition	KILLED
M3 drop the 'a==c' clause	isosceles condition	SURVIVED
M4 $a \leq 0 \rightarrow a < 0$	invalid guard boundary	KILLED
M5 $a + b \rightarrow a - b$	arithmetic operator	KILLED
M6 $b==c \rightarrow b \neq c$	equilateral condition	KILLED

THE FIX add `triangle(1,2,3)=NotATriangle` + each-clause isosceles (5,4,5),(2,3,3),(3,2,2) $\rightarrow$ mutation 4/6 (67%) $\rightarrow$ 6/6 (100%). Verified: coverage.py + a mutation harness (Python 3.10) — coverage stayed 100%, only the tests changed.

How a mutation run works

The tool generates many mutants, runs your suite against each, and scores the result.



KILLED vs SURVIVED killed = at least one test fails; survived = the whole suite still passes — a survivor is a hole in your tests.

Prerequisite: a fully green suite. A flaky or failing suite makes the mutation report meaningless.

Mutation score — and a realistic target

Killed over total (minus equivalents). Higher is better — but 100% is usually wrong.

Mutation score = $\text{killed} \div (\text{total} - \text{equivalent})$. A survivor is either a missing test or an (unkillable) equivalent mutant.



1. **70–80% is a strong target** — chasing 100% wastes time on equivalents & trivia.
2. **Exclude equivalents** — they can't be killed; count them out of the denominator.

PART 2 • THEORY

Why mutation testing works

Two 1978 assumptions still hold the field up — and they explain why tiny mutants are useful.



Where it came from

DeMillo, Lipton & Sayward — 'Hints on Test Data Selection' — proposed mutation analysis.

Competent programmer

Hypothesis programmers write nearly-correct code.

So real faults are small deviations.

Therefore small mutants stand in for real faults.

Coupling effect

Hypothesis tests that catch simple faults...

...**also catch** complex (combined) faults.

Empirically supported by later studies (Offutt).

WHY IT MATTERS these two assumptions justify using simple, single-point mutants as stand-ins for real faults.

Weak vs strong mutation

When exactly is a mutant 'killed'? The answer trades cost for strictness.

Kind	Killed when...	Trade-off
Strong	the program's final OUTPUT differs	strictest · most expensive
Weak	the STATE right after the mutated line differs	cheaper · may over-count kills
Firm	some intermediate point differs	a middle ground

IN PRACTICE most tools (Stryker, PIT) use STRONG mutation — a real test must fail — which is what you want for test quality.

First-order vs higher-order mutants

One change per mutant, or several combined — HOMs are a key cost-reduction idea.

First-order (FOM)

One change a single operator applied once.

Default what almost every tool generates.

Downside huge counts; many are trivial.

Higher-order (HOM)

Several changes combined into one mutant.

Subsuming HOMs harder to kill, fewer equivalents.

Effect far fewer mutants, similar power.

THE COUPLING EFFECT AGAIN because simple-fault tests catch complex faults, a well-chosen HOM can replace many FOMs.

The catalog of mutations

Each operator is a rule that rewrites a tiny piece of code. Each rewrite is one mutant.



Arithmetic, relational & logical

The classic operators — where most real bugs (and mutants) live.

Operator	Abbrev.	Mutation	eShop example
Arithmetic	AOR	$+$ $\rightarrow$ $-$, $*$ $\rightarrow$ $/$	<code>sub + ship</code> $\rightarrow$ <code>sub - ship</code>
Relational	ROR	$>$ $\rightarrow$ $\geq$, $==$ $\rightarrow$ $!=$	<code>stock > 0</code> $\rightarrow$ <code>stock $\geq$ 0</code>
Conditional	COR	$\&\&$ $\rightarrow$ $ $, negate	<code>loggedIn && hasItems</code> $\rightarrow$ <code> </code>
Logical / bit	LOR	$\&$ $\rightarrow$ $ $, $\wedge$	<code>flags & MASK</code> $\rightarrow$ <code>flags MASK</code>
Shortcut	SOR	short-circuit swap	<code>a ?? b</code> $\rightarrow$ <code>a b</code>

Assignment, unary, statement & constants

Deletions and constant swaps often expose the weakest assertions.

Operator	Abbrev.	Mutation	eShop example
Assignment	ASR	$+= \rightarrow -=$, = swap	<code>total += ship</code> $\rightarrow$ <code>total -= ship</code>
Unary insert	UOI	insert $++$ / $--$ / $-$	<code>qty</code> $\rightarrow$ <code>qty++</code> ; <code>x</code> $\rightarrow$ <code>-x</code>
Statement del.	SDL	remove a statement	delete 'return applyTax(t)'
Constant repl.	CRP	$0 \rightarrow 1$, $\text{true} \rightarrow \text{false}$	<code>discount = 0.1</code> $\rightarrow$ <code>0</code>
Boundary	—	$\geq \rightarrow >$, literal ± 1	<code>free-ship $\geq$ 500</code> $\rightarrow$ <code>> 500</code>

Kill the mutant — two operators

OpenDSA CS3: an Arithmetic-operator mutant and a Remove-Conditionals mutant, each killed by one assertion.

ARITHMETIC OPERATOR (AOR / AOD)

```
int Addition(int a,int b){ return a+b; }  
mutant AOD_1: return a; // survives  
mutant AOD_2: return b; // survives  
// one assertion kills BOTH  
assertEquals(15, Addition(5,10));
```

REMOVE CONDITIONALS (true / false)

```
boolean PositiveCheck(int n){  
    if (n>0) return true; else return false; }  
mutant: if(true) / if(false) -> survive  
assertTrue (PositiveCheck(10));  
assertFalse(PositiveCheck(0)); // & -5
```

THE PRINCIPLE a mutant survives until an assertion pins a value the mutation would change — pick inputs that make each branch return a distinct, asserted result.

Object-oriented operators

For OO code (Java/C#), operators target inheritance, polymorphism & access — used by MuJava / PIT.

Access (AMC)

change public $\leftrightarrow$ private $\rightarrow$ break intended encapsulation.

Hiding (IHD/IHI)

add/remove a field that hides a parent's field.

Overriding (IOD/IOR)

delete or rename an overriding method.

Polymorphism (PMD)

change a declared type to a parent/child type.

ALSO this keyword (JTD) — delete 'this.' $\rightarrow$ shadowed-variable bugs; Static (JSD/JSI) — add/remove 'static' on a member.

A few operators do most of the work

Selective mutation: a small ‘sufficient’ set approximates full mutation at a fraction of the cost.

1. **Offutt’s 5 sufficient operators** — ABS, AOR, LCR, ROR, UOI catch most of what the full set does.
2. **Why** — many operators are redundant; their mutants are subsumed by others.
3. **Payoff** — far fewer mutants to run, with little loss of test-quality signal.
4. **Tool knob** — Stryker/PIT let you enable/disable operator groups per project.

PREVIEW ‘do fewer’ is one of three cost-reduction families we return to in Part 4.

Four+ outcomes for every mutant

Not every survivor is a test gap — and not every mutant can be killed.

Killed ✓

a test failed → the suite caught the injected bug. The goal.

Survived ✗

the suite still passed → a real test gap (add an assertion).

Equivalent

code changed but behaviour identical → impossible to kill (noise).

Timeout

the mutant caused an infinite loop → counted as killed.

ALSO No coverage — the line never runs in any test → survives (write a test first); Runtime error — the mutant doesn't compile/throws → killed by the tool.

KILLED · SURVIVED · NO_COVERAGE

Valente, 'Software Engineering: A Modern Approach' — Pitest on Client.isVIP(balance) with a single weak test.

SUT + ONE WEAK TEST

```
boolean isVIP(double b){  
    if (b > 10000) return true;  
    return false; }  
// the only test  
assertTrue(isVIP(15000));
```

PIT mutator	Status
Negated conditional	KILLED
Conditional boundary	SURVIVED
Return false	KILLED
Return true	NO_COVERAGE

ONE ASSERT, ALL GREEN `assertFalse(isVIP(10000));` flips SURVIVED (boundary $>\rightarrow\geq$) and NO_COVERAGE (return-false path) to KILLED. Boundaries are where mutants hide.

The equivalent-mutant problem

Some mutants change the code but not the behaviour — no test can ever kill them.

LOOKS LIKE A GAP — ISN'T

```
// original
for (let i=0; i < items.length; i++){...}
// mutant:  $i < len \rightarrow i \neq len$ 
for (let i=0; i != items.length; i++){...}
// i increments by 1  $\rightarrow$  identical behaviour
```

Undecidable — equivalence detection is unsolvable in general.

Manual review — a human judges each stubborn survivor.

~10–20% — typical share of mutants that are equivalent.

IMPLICATION equivalent mutants are unavoidable noise — they are why 100% is unrealistic and review is mandatory.

Detecting equivalent & redundant mutants

You can't solve it, but several techniques flag likely equivalents automatically.

Technique	How	Catches
Trivial Compiler Equiv.	compile both; identical binary?	equivalent + duplicate
Compiler optimisation	dead-code / constant-fold analysis	many equivalents
Constraint / program analysis	reachability & state constraints	some equivalents
Coverage & impact	did the mutant change any output?	fast pre-filter
LLM classifiers (2024)	ask a model: same behaviour?	review-first ranking

TCE (Papadakis 2015) compile original & mutant with an optimising compiler; identical machine code $\Rightarrow$ equivalent/duplicate — free & automatic.

Redundant, duplicate & trivial mutants

Many mutants tell you nothing new — the raw score can mislead.

The problem

Duplicate different edits, same behaviour.

Subsumed killing one kills many others.

Trivial any test kills them; no signal.

The fix

Disjoint / subsuming set report only independent mutants.

Disjoint mutation score a more honest number.

De-dup with TCE remove duplicates automatically.

TAKEAWAY a raw mutation score inflated by duplicates/trivia overstates test quality — prefer disjoint-mutant reporting when available.

PART 4 · COST

Mutation testing is expensive

Thousands of mutants × a full test run each — here is how the field makes it practical.



Why it doesn't scale naively

Cost $\approx$ (# mutants) $\times$ (test-suite runtime). Both grow fast.

1. **Mutant count** — operators $\times$ mutable locations $\rightarrow$ thousands per module, millions per repo.
2. **Per-mutant run** — each mutant re-runs (part of) the suite until a test kills it.
3. **Equivalents waste time** — they never die, so the suite runs the whole way every time.
4. **Naive full mutation** — infeasible on a large codebase in CI without help.

THE ANSWER three families of techniques — do FEWER, do SMARTER, do FASTER (Offutt & Untch; Jia & Harman).

JFreeChart — the cost, and the sobering score

Valente / Rojas et al.: Pitest on JFreeChart 1.0.19 — a 47 KLOC charting library with 1,320 tests.

256K

mutants generated by Pitest

109 min

to run them all (after PIT's speed-ups)

19%

mutation score — with 1,320 tests

1. **Cost $\approx$ (#mutants) $\times$ (suite runtime)** — ~2 hours on a small 47 KLOC system shows why naive full mutation doesn't scale.
2. **A 19% score with 1,320 tests** — mutation reveals how much behaviour is executed but unchecked — the fix is incremental, not 'stop'.

Do fewer · do smarter · do faster

The standard taxonomy of mutation cost reduction.

Do fewer

- › selective mutation
- › mutant sampling
- › higher-order mutants
- › diff / changed-files only

Do smarter

- › weak mutation
- › mutant schemata (1 compile)
- › parallel / distributed
- › coverage-based skipping

Do faster

- › faster test execution
- › incremental (affected only)
- › compiler tricks (TCE)
- › caching results

Incremental, diff-based mutation

The single biggest lever in CI: only mutate the lines a change touches.

Full (batch)

Mutate the whole codebase.

When nightly / release, if at all.

Cost hours; often skipped.

Incremental (diff)

Mutate only the changed lines in a PR.

When every pull request.

Cost seconds–minutes; sustainable.

TOOLS Stryker ‘--incremental’ & diff mode; PIT ‘withHistory’ / SCM plugin — mutate the diff, not the repo.

Are mutants a valid substitute for real faults?

Just et al., FSE 2014 — the study that made practitioners take mutation seriously.

357

real faults studied (real Java programs)

73%

of real faults coupled to mutants



mutant kill correlates with real-fault
detection

WHY IT MATTERS the correlation is the empirical licence to use mutation score as a proxy for real-fault detection — the coupling effect, measured.

Mutation testing at Google

Petrović & Ivanković — how a monorepo runs mutation testing on every code review.

1. **No full score** — they don't compute a repo-wide mutation score; it isn't actionable.
2. **Diff-based, in code review** — surface a few mutants on the lines under review.
3. **Arid nodes** — suppress unproductive lines (logging, boilerplate) so mutants stay useful.
4. **~1 mutant per diff** — a low, curated dose developers actually act on.

LESSON at scale the goal is a few PRODUCTIVE mutants in review — not a big number on a dashboard.

Mutation testing on the eShop SUT

Stryker on the Node.js backend — from weak tests to killed mutants, with real output.



eShop modules worth mutating

Focus mutation effort on the risky, logic-heavy modules — not getters and DTOs.

orderTotal.js

discounts, shipping thresholds, tax — arithmetic & boundaries.

coupon.js

validity, expiry, min-spend — relational & logical operators.

inventory.js

stock checks, reservations — off-by-one & boundary bugs.

auth.js

rate-limit, roles — conditional & boundary operators.

STRATEGY skip DTOs, getters, config (trivial mutants). Start with the checkout path — it carries the money.

Survivors in the checkout total

A weak test gives 100% coverage but lets 17 of 19 mutants survive.

SUT

```
function orderTotal(items, coupon){  
  const sub = items.reduce((s,i)=>  
    s + i.price*i.qty, 0);  
  const d = coupon.pct>=10 ? .1 : 0;  
  let t = sub*(1-d);  
  return t>=500 ? t : t+30; }
```

Mutants that survive

$\geq 10 \rightarrow > 10$ — coupon boundary untested.

$1-d \rightarrow 1+d, /$ — no assert on the amount.

$\geq 500 \rightarrow > 500, <$ — free-ship boundary untested.

WHY THEY LIVE the weak test never checks the returned number, so changing the maths breaks no test.

Assertions that kill them

Each assertion pins an exact value or a boundary — now the mutants die.

FOUR ASSERTIONS → EVERY MUTANT KILLED

```
// four assertions → every mutant killed  
expect(orderTotal({price:100,qty:2},{pct:10})).toBe(210);  
expect(orderTotal({price:250,qty:2},{pct:10})).toBe(450);  
expect(orderTotal({price:300,qty:2},{pct:0})).toBe(600);  
expect(orderTotal({price:100,qty:5},{pct:9})).toBe(530);
```

A line that runs but asserts nothing still lets the mutant survive — do not inflate coverage artificially.

THE POINT killing a mutant forces a MEANINGFUL assertion — mutation testing drives better tests, not more coverage.

Different modules, different mutants

Boundaries and logic are where mutants hide — assert them explicitly.

Module · code	Mutant	Test that kills it
coupon: <code>pct ≥ 10</code>	→ <code>pct > 10</code>	a coupon with exactly <code>pct=10</code> applies
inventory: <code>stock > 0</code>	→ <code>stock ≥ 0</code>	ordering the last unit at <code>stock=0</code> fails
auth: <code>attempts ≥ 5</code>	→ <code>attempts > 5</code>	lockout on the 5th attempt, not the 6th
total: <code>round()</code>	→ <code>floor()</code>	the cent-level expected value

PATTERN for every '`≥`', '`>`', '`==`' — write a test AT the boundary. That single value kills the relational mutants.

Mutation-testing tools

Pick one main tool per language; the eShop backend uses Stryker.



Tools by language

Mature, maintained tools exist for every major stack.

Tool	Language	Style	Notes
Stryker	JS/TS/C#/Scala	source rewrite	eShop backend · HTML+JSON dashboard
PIT (PITest)	Java/Kotlin	bytecode	the standard · Maven/Gradle/JUnit
mutmut	Python	source rewrite	simple CLI · good for services
Cosmic Ray	Python	source rewrite	distributed, configurable
mull	C/C++	LLVM IR	compiles mutants natively
Infection	PHP	AST	CI-friendly, diff mode
Major / MuJava	Java	compiler/bytecode	research-grade, rich operators

Coverage & mutation tools go together

Every language has both — coverage finds untested lines; mutation checks the tests that cover them.

Language	Coverage tool	Mutation tool
Java / Kotlin	JaCoCo · Kover	PIT (Pitest)
JS / TS	Istanbul · c8 · Vitest	Stryker
Python	coverage.py + pytest-cov	mutmut · Cosmic Ray
C#	coverlet	Stryker.NET
Rust	cargo-llvm-cov	cargo-mutants

WORKFLOW coverage finds untested lines → you add tests → mutation proves those tests actually assert — run both in CI.

Stryker (JS) & PIT (Java)

The two you'll meet most — how they run and what they give you.

Stryker

Config stryker.conf.json: testRunner, mutate globs.

Runners Jest, Mocha, Vitest, Karma, Cucumber.

Reports clear-text, HTML, JSON, dashboard.

PIT (PITest)

Integrates Maven / Gradle plugin + JUnit/TestNG.

Bytecode fast; no source recompile per mutant.

Reports HTML with line-level mutant status.

THIS SEMINAR install Stryker on the eShop backend, run a baseline, then hunt survivors in the checkout module.

LIVE DEMO

Stryker on eShop — real output

A tiny orderTotal.js + Jest. Weak test first, then strong — watch the score move.



Three files, three commands

Everything below is a real, reproducible run — no slideware.

TERMINAL

```
# 1) install
npm i -D jest @stryker-mutator/core @stryker-mutator/jest-runner
# 2) stryker.conf.json
{ "testRunner": "jest", "reporters": ["clear-text", "html", "json"], "mutate": ["orderTotal.js"] }
# 3) run
npx stryker run
```

WHAT STRYKER DOES rewrites orderTotal.js into ~19 mutants, runs Jest against each, and reports killed vs survived per line.

Weak test → 10.53%

One assertion-free test: 100% coverage, but 17 of 19 mutants survive. Real output:

`npx stryker run (weak test)`

```
[Survived] ArithmeticOperator orderTotal.js:4:17  
- const total = sub * (1 - d);  
+ const total = sub * (1 + d);  
[Survived] EqualityOperator orderTotal.js:6  
Mutation score: 10.53% (2/19 killed)
```

What it means

100% coverage — yet only 2 mutants die.

17 survivors — every one is a missing assertion.

Score 10.53% — the suite barely tests anything.

READ IT coverage says 'done'; mutation says 'you've tested almost nothing'. The gap is the whole point.

Strong tests → 100%

Add four value/boundary assertions; re-run. Every mutant now dies. Real output:

`npx stryker run (strong tests)`

```
All files    100.00 |  
  #killed 19  #survived 0  #timeout 0  
// same code, better tests  
// coverage was already 100%  
// mutation score: 10.53% → 100%
```

The headline

Coverage didn't change — it was 100% the whole time.

Mutation score is what actually improved.

THE HEADLINE coverage didn't move; mutation score did — that is the signal coverage cannot give you.

AI drafts, the mutation report judges

LLMs are strong at proposing killing assertions and flagging equivalents — you verify.



Four AI angles on mutation testing

Use AI to cut the manual toil — but never to decide; the report decides.

Kill survivors

LLM: function + surviving mutant → candidate assertions.

Generate mutants

LLMs propose natural, diverse mutants (LLMorpheus, μ BERT).

Flag equivalents

classify likely-equivalent survivors to review first.

Prioritise

rank mutants by risk / productivity to show few, useful ones.

THE RULE AI drafts (incl. DiffBlue Cover's JUnit targeting un-killed mutants); a real mutation run must confirm each kill.

The AI coverage trap

LLM-generated tests can inflate coverage while checking nothing — mutation score is the guard.

The trap

Assertion-free tests execute code, assert nothing, pass trivially.

Coverage jumps bug-detection does not.

The tell coverage 62%→91%, mutation 58%→60%.

The guard

Review require meaningful assertions; reject calls-without-checks.

Mutation gate score the AI suite by mutation, not coverage.

Rule a passing suite that kills nothing is worthless.

BOTTOM LINE an LLM can hit any coverage target while checking nothing — only a mutation run exposes the padding.

LLM assertion synthesis for a survivor

Give the model the code + the surviving mutant; verify its assertions against Stryker.

AI-FIRST WORKFLOW

// PROMPT to the LLM

"Here is orderTotal(). Stryker reports this survivor:

`total >= 500` → `total > 500`.

Write ONE Jest assertion that KILLS it."

→ `expect(orderTotal([price:250,qty:2]),{pct:0}).toBe(500);`

VERIFY, ALWAYS LLM assertions can be wrong or assert the mutated value — only a green mutation run proves a kill. In our run, AI-style boundary asserts took the score 10.5% → 100% — after verification.

LLMs for mutation testing – the landscape

A fast-moving area; promising, but validation against the report is still non-negotiable.

Work	Idea	Finding
LLMorpheus (2024)	LLM generates mutants from code	more natural, diverse mutants
μBERT	masked-LM produces mutants	competitive with rule-based
LLMs for equiv. detection	ask model: same behaviour?	helps, but not reliable alone
Mutahunter	LLM-driven, language-agnostic	context-aware mutants

CAUTION LLMs can produce equivalent or trivial mutants and over-confident assertions — measure, don't trust.

Putting it in CI/CD

Make mutation testing sustainable and actionable, not a nightly report nobody reads.

1. **Gate on the diff** — run incremental mutation on changed files in each PR.
2. **Set a threshold** — fail the build if the diff's mutation score drops below a bar.
3. **Show, don't dump** — surface a few survivors inline in the review — not a giant report.
4. **Exclude the noise** — filter equivalents / arid nodes; track a disjoint score.

BALANCE a small, fast, diff-scoped run every PR beats a giant nightly score no one acts on.

Failure modes — how it can mislead

Powerful but noisy and slow; know the traps.

Equivalent noise

survivors that can't be killed inflate the 'gap' count — review, don't chase.

Slow & costly

mutants × full test run; use incremental / changed-files mode.

Score gaming

killing only easy mutants with trivial asserts games the number.

Not correctness

a high score means sensitive tests, not a bug-free program.

ALSO needs a green suite (flaky tests make the report meaningless); trivial mutants — prefer disjoint-mutant reporting.

Kill the Mutant

Turn a survivor into a meaningful assertion — hands on the eShop checkout module.

The activity

5 surviving mutants from the eShop checkout module.

Each team writes 1 assertion per mutant (Jest / pytest).

Pair-swap: would these assertions actually kill the mutant?

Re-run Stryker; confirm the score moves.

Takeaway

Coverage = ran. Mutation = caught.

Target 70–80%. Equivalents = noise.

Do fewer / smarter / faster.

AI drafts, you verify.

Reading: Jia & Harman (2011) · Just et al. (FSE 2014) · Petrović & Ivanković — Mutation Testing at Google · Stryker/PIT docs.